

Assignment 10.3

M.koushik reddy
2103a52153

Problem Statement 1: AI-Assisted Bug Detection

```
def factorial(n):  
    if n < 0:  
        return "Invalid input"  
  
    result = 1  
    for i in range(1, n + 1):  
        result *= i  
  
    return result  
  
print(factorial(5))
```

Output

120

Explanation

The AI found the mistake in the loop and fixed it by changing the range to include `n`. It also handled edge cases like

negative numbers and zero. The manual fix only corrected the loop, but the AI solution made the program safer by adding input checking. So, the AI did not miss important edge cases.

Task 2 — Improving Readability & Documentation

```
def calculate_numbers(num1, num2, operation):  
    """  
    Performs a basic arithmetic operation on two numbers.  
  
    Parameters:  
    num1 (int or float): First number  
    num2 (int or float): Second number  
    operation (str): Operation to perform ("add", "sub", "mul", "div")  
  
    Returns:  
    int/float or str: Result of the operation or error message  
  
    Example:  
    calculate_numbers(10, 5, "add") -> 15  
    """  
  
    if not isinstance(operation, str):  
        return "Invalid operation type"  
  
    if operation == "add":  
        return num1 + num2  
    elif operation == "sub":  
        return num1 - num2  
    elif operation == "mul":  
        return num1 * num2  
    elif operation == "div":  
        if num2 == 0:  
            return "Cannot divide by zero"  
        return num1 / num2  
    else:
```

```
        return "Unknown operation"

# Test cases
print(calculate_numbers(10, 5, "add"))
print(calculate_numbers(10, 5, "div"))
print(calculate_numbers(10, 0, "div"))
print(calculate_numbers(10, 5, 123))
```

Output

15

2.0

Cannot divide by zero

Invalid operation type

Task 3: Enforcing Coding Standards

```
def check_prime(n):
    """Returns True if a number is prime, otherwise False."""

    if n <= 1:
        return False

    for i in range(2, n):
        if n % i == 0:
            return False

    return True
```

```
print(check_prime(7))
print(check_prime(10))
```

Output

True

False

Task 4: AI as a Code Reviewer in Real Projects

```
from typing import List, Union

def double_even_numbers(numbers: List[Union[int, float]]) ->
List[Union[int, float]]:
    """
    Returns a list where all even numbers are doubled.

    Parameters:
    numbers (list): List of numeric values

    Returns:
    list: List containing doubled even numbers
    """

    if not isinstance(numbers, list):
        raise TypeError("Input must be a list")

    result = []
    for num in numbers:
        if isinstance(num, (int, float)) and num % 2 == 0:
            result.append(num * 2)
```

```
    return result

print(double_even_numbers([1, 2, 3, 4, 5, 6]))
```

Output

[4, 8, 12]

Reflection

AI should be used as an assistant in code reviews rather than a standalone reviewer. It can quickly identify readability issues, missing validation, and suggest improvements. However, human developers are still important for understanding project requirements, logic correctness, and design decisions. Combining AI assistance with human review makes code reviews faster and more reliable.

Task 5: — AI-Assisted Performance Optimization

Optimized Version (AI-suggested)

```
def sum_of_squares_optimized(numbers):
    return sum(x * x for x in numbers)
```

Performance Comparison

```
import time

nums = range(1000000)

start = time.time()
sum_of_squares(nums)
print("Original time:", time.time() - start)

start = time.time()
sum_of_squares_optimized(nums)
print("Optimized time:", time.time() - start)
```

Example output

Original time: 0.09

Optimized time: 0.06

Trade-off Discussion:

Both implementations have the same $O(n)$ time complexity, but the optimized version is shorter and faster due to Python's built-in functions. The original version is easier for beginners to understand, while the optimized version improves performance and code readability. NumPy can provide even better performance for very large datasets, but it adds an external dependency.

Execution Time Comparison:

After testing both functions with a large dataset (`range(1000000)`), the optimized version runs slightly faster than the original version. This is because the optimized function uses Python's built-in `sum()` function, which is internally optimized and more efficient than a manual loop.

Performance Improvements Suggested by AI:

Performance can be improved by using Python's built-in `sum()` function with a generator expression instead of a manual loop. Built-in functions are optimized and usually execute faster.

For very large datasets, NumPy vectorization can also be used. NumPy performs calculations using optimized numerical operations, which can be faster than standard Python loops.

Example improvements:

Use `sum(x * x for x in numbers)` instead of a loop.

Use NumPy arrays for large numerical computations.

